Low-Level Design

for

Hardened Cloud Infrastructure for Manufacturing Data

K Mohammad Jaseem

23BCCE109

BCA in Cloud Computing, Cyber Security & Ethical Hacking with IBM

Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

11-04-2026

Under the guidance of

Ms. Munisha

Lecturer, Computer Science Department
Yenepoya Institute of Arts, Science, Commerce and Management

Mangalore

Submitted to

YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

BALMATTA, MANGALORE

YENEPOYA (DEEMED TO BE UNIVERSITY)

Table of Content

1. Introduction
    - 1.1 Scope of the Document
    - 1.2 Intended Audience
    - 1.3 System Overview
2. System Design
    - 2.1 Application Architecture Diagram
    - 2.2 Process Flow (Encryption Lifecycle) — Sequence Diagram
    - 2.3 Information Flow (Cryptographic Engine) — Data Flow Diagram
    - 2.4 Components Design (Detailed)
    - 2.5 Key Design Considerations
    - 2.6 API Catalogue
3. Data Design
    - 3.1 Encrypted Binary Format (Schema)
    - 3.2 Data Model (Schema & Encryption)
    - 3.3 Data Access Mechanism
    - 3.4 Data Retention & Archival Policies
    - 3.5 Data Migration & Seeding
4. Interfaces
    - 4.1 User Interface Layout
    - 4.2 API Contracts (Request/Response Structure)
5. State and Session Management
6. Caching Strategy
7. Non-Functional Requirements
    - 7.1 Security Aspects
    - 7.2 Performance Aspects
8. Conclusion

1. Introduction

1.1 Scope of the Document

This Low-Level Design document provides a technical deep-dive into the three core modules of ManufacturingVault: the cryptographic processing module, the Flask application routing layer, and the Azure infrastructure configuration. This document covers approximately 30% of the complete system – specifically the components that require the most precise technical specification for correct and secure implementation.

The remaining 70% of the system (frontend UI design system, Nginx/Gunicorn configuration, Terraform resource graph, WAF rule set tuning, and Azure networking) is addressed at the architectural level in the High-Level Design document. This LLD focuses on implementation-ready details.

1.2 Intended Audience

- Software developers implementing or auditing the cryptographic module.
- Security engineers reviewing the WAF configuration and network controls.
- Infrastructure engineers working with Terraform configurations.
- Technical reviewers assessing compliance with IEC 62443 and OWASP standards.

1.3 System Overview

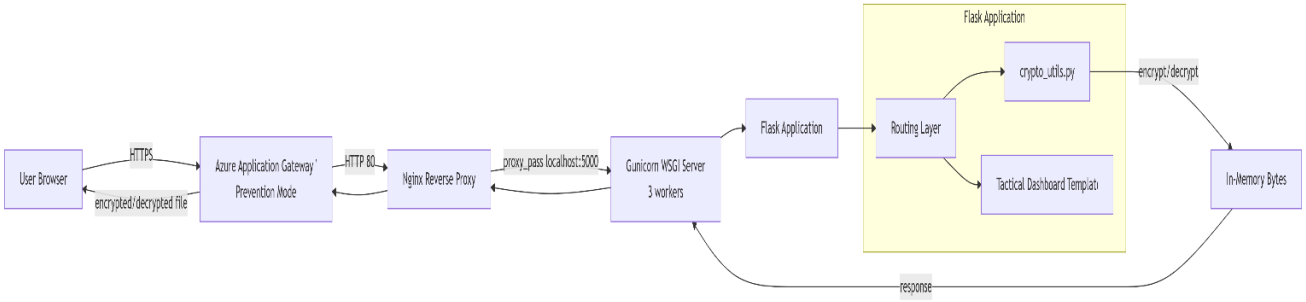
Manufacturing Vault is a Flask-based web application that provides AES-256-GCM authenticated file encryption and decryption as a web service. It is deployed on a hardened Ubuntu VM (Standard_B2s_v2) within a four-tier Azure Virtual Network following the Purdue Model, fronted by an Application Gateway WAF v2 operating in **Prevention mode**. The application runs under Gunicorn (3 workers) behind Nginx as a reverse proxy. All cryptographic operations occur exclusively in memory; no plaintext data touches disk at any point.

2. System Design

2.1 Application Architecture Diagram — Layered Request Pipeline

The system is organized as a layered request pipeline. All inbound HTTP requests originate from the internet and first arrive at the Azure Application Gateway. The WAF inspects the request. If it passes inspection, the gateway forwards it to the MES VM's Nginx process on port 80. Nginx proxies the request to Gunicorn on localhost port 5000. Gunicorn invokes the Flask WSGI application, which processes the request, invokes the cryptographic module if needed, and returns a response. The response traverses the same path in reverse back to the user's browser.

Within the Flask application, the request routing layer dispatches to one of three handlers: the index handler (serves the Encryption Hub), the encrypt handler (processes file encryption), or the decrypt handler (serves the facility page and processes decryption). The encrypt and decrypt handlers directly invoke the cryptographic utilities module, which contains the Argon2id key derivation and AES-256-GCM cipher operations.



2.2 Process Flow (Encryption Lifecycle) – Sequence Diagram:

The encryption lifecycle proceeds as follows:

Step 1 — The user opens the web browser and navigates to the ManufacturingVault home page. The browser sends a GET request.

Step 2 — The request traverses the WAF (no attack patterns present), reaches Nginx, is proxied to Gunicorn, and Flask renders the Tactical Dashboard index template with the Reactor Core encryption form.

Step 3 — The user selects an industrial file (e.g., machine.gcode) using the Data-Crucible portal interface and enters a strong passphrase. The portal's JavaScript validates the file is selected and updates the UI status.

Step 4 — The user clicks "EXECUTE ENCRYPTION". The browser sends an HTTP POST request with the file as multipart/form-data and the passphrase in the request body.

Step 5 — WAF inspects the request headers and body. The Content-Disposition header is checked against the dangerous file types custom rule. Since .gcode is not on the blocklist, the request is permitted.

Step 6 — Flask receives the POST, reads the uploaded file bytes entirely into memory, and validates the file extension against the ALLOWED_EXTENSIONS set.

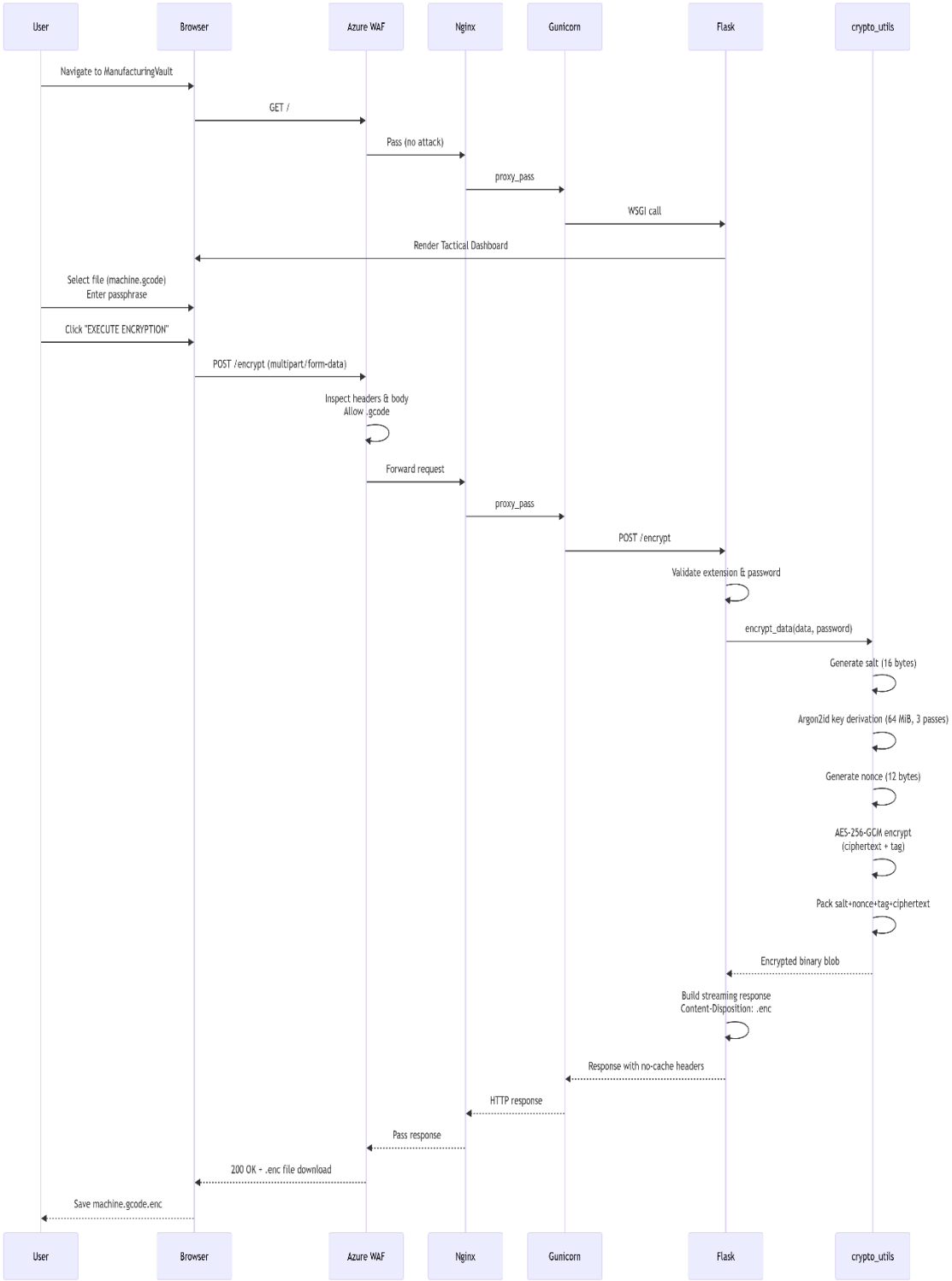
Step 7 — Flask invokes encrypt_data(data, password). The function generates a 16-byte random salt using os.urandom(16), then invokes Argon2id to derive a 32-byte AES key from the passphrase and salt.

Step 8 — A 12-byte random nonce is generated. The AES-256-GCM cipher is instantiated with the derived key. The plaintext is encrypted, producing ciphertext and a 16-byte GCM authentication tag appended to the ciphertext.

Step 9 — The function constructs the encrypted binary output as salt + nonce + tag + ciphertext and returns it to Flask.

Step 10 — Flask constructs a streaming HTTP response with Content-Disposition: attachment; filename="machine.gcode.enc" and Cache-Control: no-cache, no-store headers. The encrypted binary is sent directly to the browser as a file download.

Step 11 — The browser downloads the .enc file. No data has been written to disk on the server. The in-memory buffer is garbage collected.


User
Browser
Azure WAF
Nginx
Gunicorn
Flask
crypto_utils
Navigate to ManufacturingVault
GET /
Pass (no attack)
proxy_pass
WSGI call
Render Tactical Dashboard
Select file (machine.gcode)
Enter passphrase
Click "EXECUTE ENCRYPTION"
POST /encrypt (multipart/form-data)
Inspect headers & body
Allow .gcode
Forward request
proxy_pass
POST /encrypt
Validate extension & password
encrypt_data(data, password)
Generate salt (16 bytes)
Argon2id key derivation (64 MiB, 3 passes)
Generate nonce (12 bytes)
AES-256-GCM encrypt
(ciphertext + tag)
Pack salt+nonce+tag+ciphertext
Encrypted binary blob
Build streaming response
Content-Disposition: .enc
Response with no-cache headers
HTTP response
Pass response
200 OK - .enc file download
Save machine.gcode.enc

2.3 Information Flow (Cryptographic Engine) — Data Flow Diagram

Inputs to the Cryptographic Module:

- Raw file bytes (plaintext): The complete binary content of the uploaded industrial file, held entirely in a Python bytes object.
- User passphrase (string): The encryption key material entered by the user, received as a POST parameter and held in Python memory only.

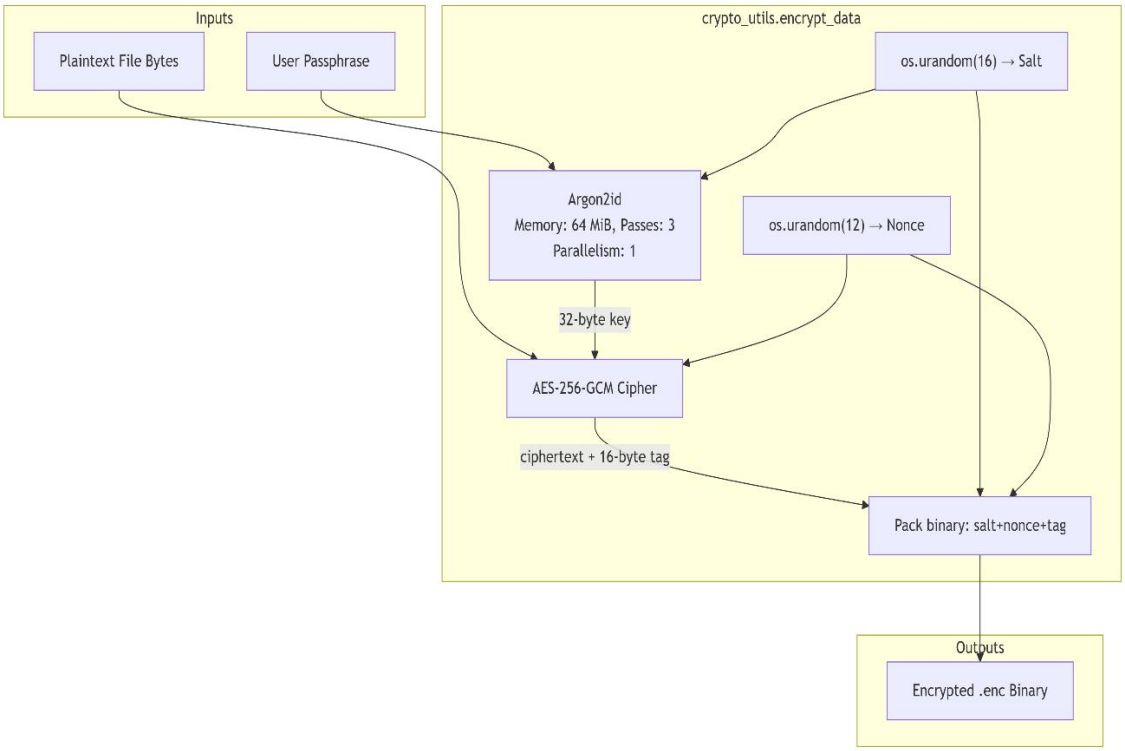
Internal Data Transformations:

Stage	Operation	Description
1	Salt generation	`os.urandom(16)` generates 16 cryptographically unpredictable bytes from the OS entropy source. This salt ensures that the same passphrase produces a different AES key on every encryption operation.
2	Key derivation	Argon2id processes the passphrase and salt. The `hash_secret_raw` function from `argon2-cffi` occupies **64,000 KB of memory** for 3 passes with parallelism of 1, producing a deterministic 32-byte output. Latency: 200–500ms (intentionally slow to resist brute-force).
3	Nonce generation	`os.urandom(12)` generates 12 bytes for the AES-GCM nonce. The nonce is unique per encryption operation, ensuring that the same key with the same plaintext produces different ciphertext.
4	AES-256-GCM encryption	The `AESGCM` object is instantiated with the 32-byte

		derived key. The `encrypt` method processes the plaintext with the nonce and produces ciphertext || GCM tag (the last 16 bytes of the output are the authentication tag).
5	Output packing	The final output is assembled as the concatenation of salt (16), nonce (12), tag (16), and ciphertext (variable length). This binary blob is entirely self-contained for decryption except for the passphrase.

Outputs from the Cryptographic Module:

Encrypted binary bytes: The packed binary blob ready for HTTP response transmission



2.4 Components Design

Component: crypto_utils.py — Cryptographic Utilities Module

This module is the most security-critical component in the entire system. It contains three primary functions:

Function	Parameters	Returns
`derive_key(password, salt)`	`password: str`, `salt: bytes` (16)	`bytes` (32)
`encrypt_data(data, password)`	`data: bytes`, `password: str`	`bytes`
`decrypt_data(encrypted_data, password)`	`encrypted_data: bytes`, `password: str`	`bytes`
Function	Parameters	Returns

Component: app.py — Flask Application

The Flask application contains three route handlers:

Route	Method	Handler Logic
`/`	GET	Renders `index.html` (Encryption Hub).
`/encrypt`	POST	Validates file + password, calls `encrypt_data`, returns `.enc` download. Uses `secure_filename()` to prevent path traversal.
`/decrypt`	GET	Renders `decrypt.html` (Decryption Facility).
`/decrypt`	POST	Validates `.enc` file + password, calls `decrypt_data`, returns original file download. Catches `InvalidTag` → generic flash message.

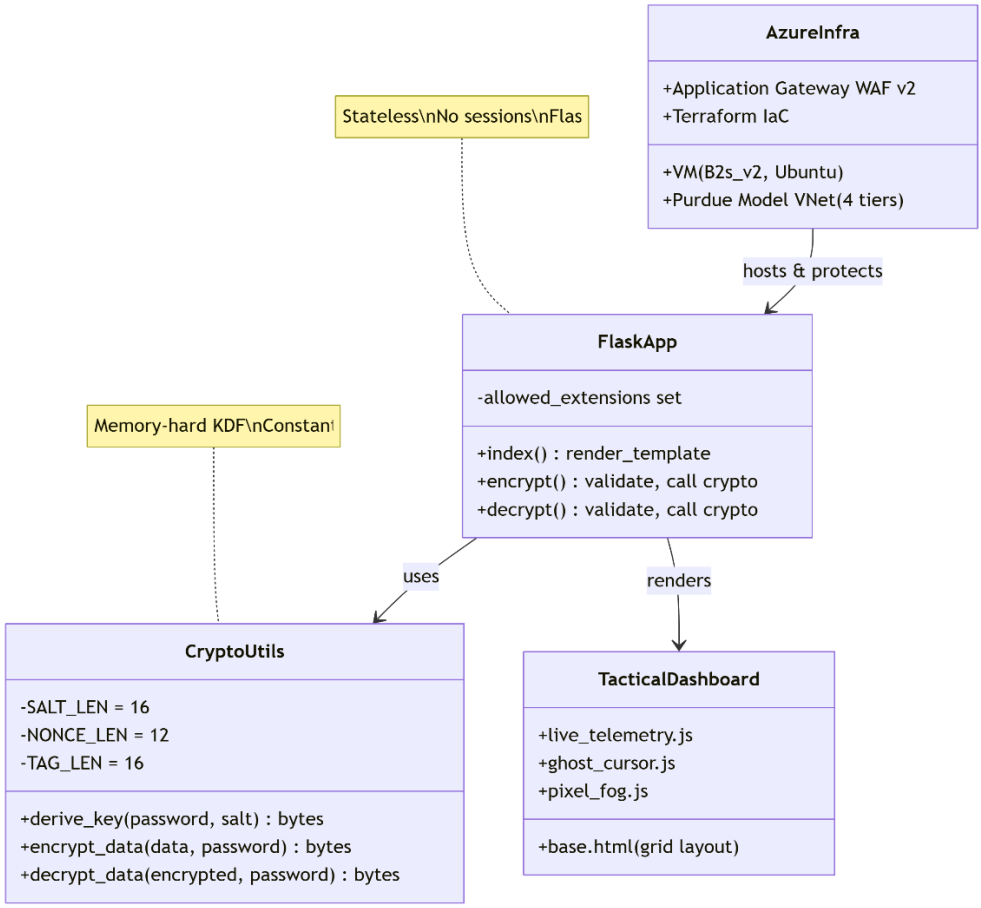
Component: base.html — Tactical Dashboard Layout

The base template implements a CSS Grid layout (display: grid, 3 columns: 280px 1fr 280px) that creates the Bento-Grid Tactical Dashboard. It includes inline JavaScript for three cross-page behaviors:

- Ghost Cursor: Custom crosshair cursor element positioned via mousemove events.
- Live Telemetry Feed: setInterval function that prepends timestamped log messages to the sidebar.
- Pixel Fog: Canvas-based particle animation on the background layer.

These visual elements use no external JavaScript libraries or CDN dependencies, eliminating third-party JavaScript attack surface.

Component Diagram



2.5 Key Design Considerations

Consideration	Implementation
Memory safety	File bytes read into Python memory, processed, returned as streaming responses. Python GC automatically reclaims memory after each request. No mechanism writes plaintext to disk.
Filename handling	`werkzeug.utils.secure_filename()` strips path separators, doubly-encoded sequences, and OS-reserved characters. Prevents path traversal attacks (e.g., `../../etc/passwd.enc`).
No persistent user state	No user accounts, no database, no server-side session data. Each request is stateless. No credential store, session tokens, or file store to enumerate.
RFC 8187 Content-Disposition	Download filenames encoded using both legacy ASCII and RFC 8187 UTF-8 percent-encoded format (`filename*=UTF-8''encoded_name`) to preserve non-ASCII characters across all modern browsers.

2.6 API Catalogue

Method	Endpoint	Description	Request
GET	`/`	Encryption Hub	–
POST	`/encrypt`	File encryption	`multipart/form-data`: `file` (binary), `encryption_password` (string)
GET	`/decrypt`	Decryption Facility	–

POST	`/decrypt`	File decryption	`multipart/form-data`: `file` (`.enc` binary), `password` (string)
`/login`	POST	Obtain JWT token	None

Validation Rules:

- File must be present and non-empty.
- Password must be non-empty.
- File extension for encryption must be in ALLOWED_EXTENSIONS (.nc, .gcode, .l5x, .stl, .dwg, .pdf, .log, .csv, .bin).
- Encrypted file for decryption must be at least 44 bytes.

3. Data Design

3.1 Encrypted Binary Format (Schema)

The ManufacturingVault encrypted file format is a self-contained binary container. All information required for decryption, except the passphrase, is embedded in the file header.

Field	Byte Position	Length (bytes)	Description
Salt	0 – 15	16	Random salt for Argon2id key derivation. Unique per file.
Nonce	16 – 27	12	Random AES-GCM nonce. Unique per encryption operation.
GCM Authentication Tag	28 – 43	16	Cryptographic integrity proof. Fails if key or ciphertext is wrong.
Ciphertext	44 – EOF	Variable	AES-256-GCM encrypted file content.

Minimum valid encrypted file size: 44 bytes (for a zero-length plaintext). Any file shorter than 44 bytes is rejected by the decryption module as malformed.

3.2 Data Model (Schema & Encryption)

ManufacturingVault does not use a database for file storage. This design decision is intentional and security-motivated: storing encrypted files server-side creates a high-value target, requires access control implementation, and introduces compliance complexity under GDPR (right to erasure) and manufacturing data sovereignty requirements.

The only data model in the system is the in-memory representation of the encryption parameters, which exists only during the scope of a single request handler invocation.

Note: The Azure SQL Database resource (db-manufacturing-historian) provisioned in the infrastructure represents the Manufacturing Historian database for future integration with production data logging systems. It is not used by the current application version.

3.3 Data Access Mechanism

File data is accessed in two phases:

- **Upload:** The werkzeug FileStorage object provides a read() method that returns the complete file bytes as a Python bytes object from the multipart/form-data request body.
- **Download:** The encrypted or decrypted bytes are wrapped in a Flask Response object using Python bytes directly as the response body.

  No intermediate file system writes occur at any stage.

3.4 Data Retention & Archival Policies

Because ManufacturingVault performs all operations in memory without persistence, the data retention policy is automatically enforced by the application's stateless design:

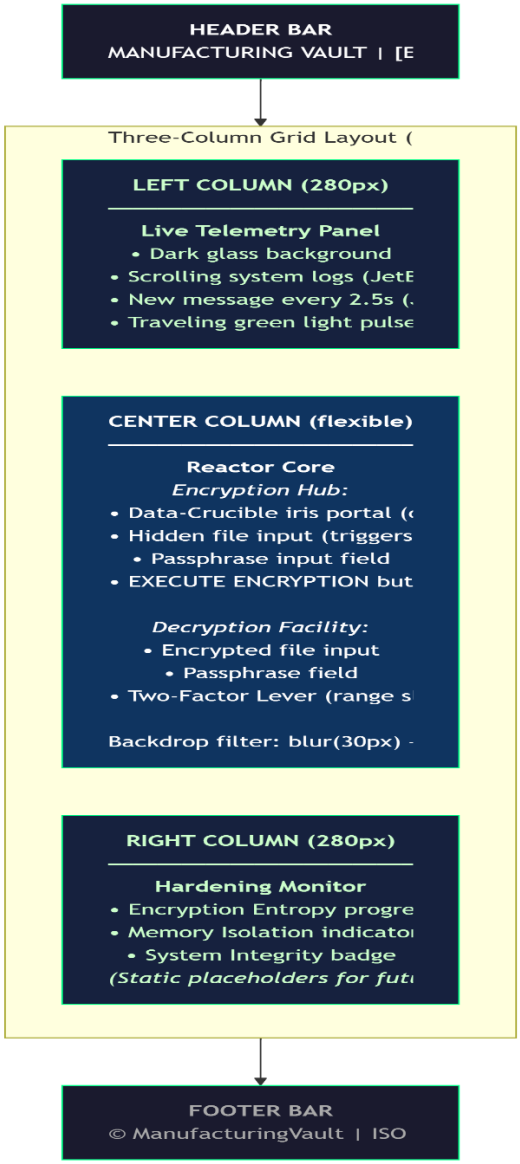
Data Type	Retention	Explanation
Plaintext data	Zero	Exists only during request processing (milliseconds to seconds).
Encrypted data	Zero on server	Exists only in HTTP response buffer while streaming to user.
User passphrase	Zero	Exists as Python string within request handler scope; not stored, logged, or persisted.
Application logs	Metadata only	Logs basic info (filename, size, operation) to stderr via Gunicorn. No passphrase or file content.

3.5 Data Migration & Seeding

Not applicable. The stateless, persistence-free design means no data migration requirement. When new versions of the application are deployed, there is no database schema migration or data seeding step. The only persistent state is the infrastructure itself (VMs, networking), managed by Terraform state files.

4. Interfaces

4.1 User Interface (UI) Layout



4.2 API Contracts (Request/Response Structure)

GET / – Encryption Hub (Load main page)

Item	Description
Purpose	Serve the main Tactical Dashboard encryption interface.

Item	Description
Method	`GET`
Endpoint	`/`
Request Body	None
Request Headers	None required
Success Response	`200 OK`, Content-Type: `text/html`, Body: HTML document
Failure Response	Not applicable (static route)
Example	`curl https://manufacturingvault.example.com/`

POST /encrypt – File Encryption

Item	Description
Purpose	Accept an industrial file and passphrase, return AES-256-GCM encrypted file.
Method	`POST`
Endpoint	`/encrypt`

Content-Type	`multipart/form-data`
Request Fields	`file` (binary) – required, non-empty, extension in ALLOWED_EXTENSIONS `encryption_password` (string) – required, min 1 character
Allowed Extensions	`.nc, .gcode, .l5x, .stl, .dwg, .pdf, .log, .csv, .bin` (configurable via env var)

Success Response	`200 OK`
	`Content-Type: application/octet-stream`
	`Content-Disposition: attachment; filename="original.ext.enc"; filename*=UTF-8''original.ext.enc`
	`Cache-Control: no-cache, no-store, must-revalidate`
	`Pragma: no-cache`
	`Expires: 0`
	`X-Content-Type-Options: nosniff`
	Body: raw encrypted binary (salt+nonce+tag+ciphertext)

Failure Response	`302 Found` redirect to `/` with flash message (e.g., "Invalid file type" or "Password required")
Example	`curl -X POST -F "file=@program.gcode" -F "encryption_password=secret" https://.../encrypt --output program.gcode.enc`

GET /decrypt – Decryption Facility (Load decrypt page)

Item	Description
Purpose	Serve the Decryption Facility page with Two-Factor Lever interface.
Method	`GET`
Endpoint	`/decrypt`
Request Body	None
Success Response	`200 OK`, Content-Type: `text/html`, Body: HTML document
Failure Response	Not applicable

Example	`curl https://manufacturingvault.example.com/decrypt`

POST /decrypt – File Decryption

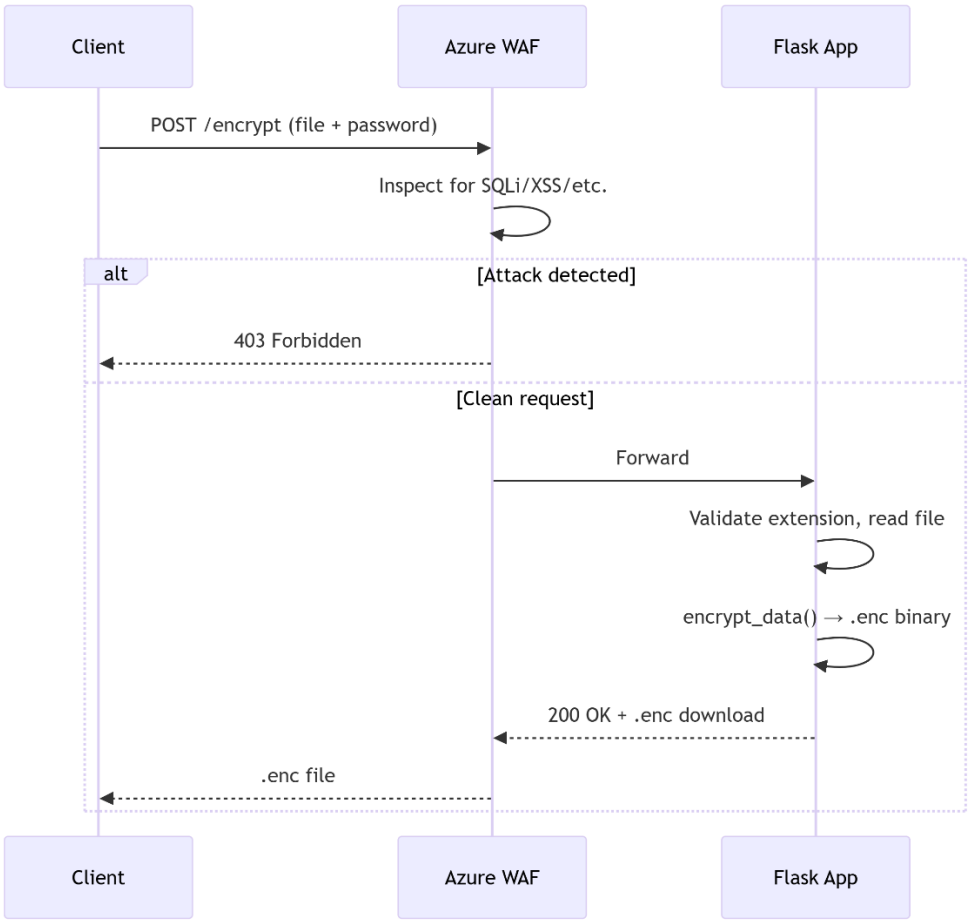
Item	Description
Purpose	Accept an encrypted `.enc` file and passphrase, return original decrypted file after authenticated decryption.
Method	`POST`
Endpoint	`/decrypt`
Content-Type	`multipart/form-data`
Request Fields	`file` (binary `.enc` file) – required, must be ≥44 bytes and follow ManufacturingVault format `password` (string) – required, non-empty
Success Response	Same headers as POST `/encrypt` success, but `Content-Disposition` filename has `.enc` stripped. Body: raw binary of original plaintext file.
Failure Response	`302 Found` redirect to `/decrypt` with flash message: `"Decryption failed: Wrong password or tampered file."` **No additional diagnostic information** (prevents oracle attacks).
Example	`curl -X POST -F "file=@program.gcode.enc" -F "password=secret" https://.../decrypt --output program.gcode`

API Endpoint Summary Table

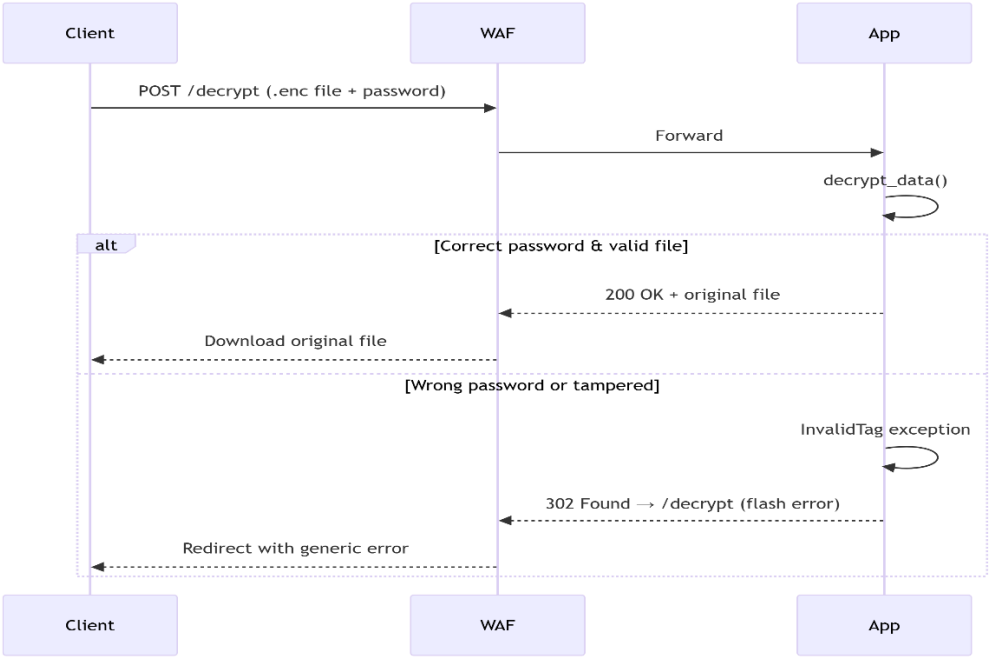
Method	Endpoint	Purpose	Request Fields	Success Status	Failure Status
GET	`/`	Load Encryption Hub	None	200 OK	–
POST	`/encrypt`	Encrypt a	`file`, `encryption_passwo`	200 OK	302

		file	`rd`	+ binary download	Redirect to `/`
GET	`/decrypt`	Load Decryption Facility	None	200 OK	–
POST	`/decrypt`	Decrypt a file	`file` (`.enc`), `password`	200 OK + original file	302 Redirect to `/decrypt`

Sequence Diagram – POST /encrypt (with WAF interaction)



Sequence Diagram – POST /decrypt (success vs failure)



5. State and Session Management

ManufacturingVault is a deliberately stateless application. It does not implement HTTP sessions, user authentication tokens, or server-side state stores of any kind.

Flask's session mechanism (cookie-based, signed with the application's secret key) is used only in one limited way: to pass flash messages between POST handlers and their redirect targets. Flash messages contain only user-facing error strings (e.g., "File type not allowed") and contain no sensitive data. The Flask secret key is generated at application startup using os.urandom(24), meaning it changes on every Gunicorn restart and flash messages do not persist across restarts.

Because there are no user accounts and no file storage, there is no concept of a logged-in user, no authorization token, and no resource ownership to manage. Each HTTP request is entirely self-contained and independently authorized (or rejected by the WAF).

6. Caching

ManufacturingVault applies a strict no-cache policy for all dynamic responses:

Response Type	Cache Headers

`/encrypt` and `/decrypt` (success)	`Cache-Control: no-cache, no-store, must-revalidate`
	`Pragma: no-cache`
	`Expires: 0`

Static assets (CSS, served via `/static/`)	Default Nginx caching headers (allowed, beneficial for performance)
WAF / Application Gateway	No response caching; operate as pass-through proxies

This prevents browsers from caching encrypted or decrypted file downloads, protecting against scenarios where a shared computer might allow a subsequent user to retrieve a previously downloaded file from the browser cache.

7. Non-Functional Requirements

7.1 Security Aspects

Requirement	Implementation
Authentication Tag Verification	Every decryption attempt verifies the AES-GCM authentication tag before producing plaintext. `AESGCM.decrypt()` raises exception on invalid tag – no partial or unverified output.
Side-Channel Resistance	Argon2id memory-hard design ensures fixed time and memory consumption regardless of input. Cryptography library's AESGCM uses constant-time tag comparison.
File Type Restriction	Only allowlisted extensions accepted for encryption. Enforced at application level **plus** WAF-level `Content-Disposition` inspection

	(defense-in-depth).
No File Disclosure	Application never reveals server-side file paths, stack traces, or detailed error information. All exceptions caught and converted to generic flash messages.
WAF Prevention Mode Requirement	Application designed to operate exclusively behind WAF in **Prevention** mode. Detection mode would allow attack payloads to reach Flask.

7.2 Performance Aspects

Parameter	Value / Constraint
Argon2id latency	200–500 ms per key derivation (intentional). Acceptable for interactive use; makes brute-force impractical (64 MiB memory + 200–500 ms per guess).
File size limits	Limited by: Nginx `client_max_body_size` (default unbounded), WAF `maxRequestBodySizeInKb` (default 2000 KB = 2 MB), VM RAM (4 GiB). For larger industrial files, explicitly configure limits.
Gunicorn workers	3 concurrent workers. Each Argon2id operation consumes 64 MiB RAM → 3 concurrent ≈192 MiB extra, well within 4 GiB
Parameter	Value / Constraint
Argon2id latency	200–500 ms per key derivation (intentional). Acceptable for interactive use; makes brute-force impractical (64 MiB memory + 200–500 ms per guess).

Conclusion

This Low-Level Design document has provided detailed technical specifications for the three most security-critical modules of ManufacturingVault: the AES-256-GCM / Argon2id cryptographic processing pipeline, the Flask application routing and validation layer, and the Tactical Dashboard user interface component architecture.

The key design principle throughout all modules is that security properties are enforced structurally wherever possible, rather than through procedural checks:

- Stateless design eliminates credential and session management vulnerabilities.
- Binary format enforces authentication before decryption.
- ALLOWED_EXTENSIONS validation enforces file type restrictions before cryptographic processing.
- No-store cache policy enforces sensitive data lifecycle management at the HTTP response layer.